CV Intern Task — Report

Egg Segmentation and Tracking

Tasks 1-4: Pretrained Segmentation, Classical Segmentation, Evaluation, and Video Tracking

Section 1: Task 1 - Pretrained-Model Egg Segmentation

1.1 Objective

The goal of Task 1 was to segment individual eggs in a dataset of real-world photos using pretrained, off-the-shelf models — no custom training or fine-tuning. The output of this task (instance masks) serves two purposes downstream: it is treated as **ground truth** for evaluating the classical algorithm built in Task 2 (see Task 3), and it establishes the dev/eval split reused throughout the project.

1.2 Pipeline

A two-stage, zero-shot detect-then-segment pipeline was used:

1. **Detection — OWL-ViT (open-vocabulary object detector).** Each image is passed through OWL-ViT with the text prompt `"egg"`. OWL-ViT is a text-prompted detector, able to localize objects matching a natural-language description without being trained specifically on eggs. This produces bounding boxes around candidate egg regions, with confidence scores.
2. **Segmentation — SAM (Segment Anything Model), box-prompted.** Each OWL-ViT bounding box is passed as a prompt into SAM, which produces a precise pixel-level instance mask for the object inside that box.

This combination allows fully automatic, label-free instance segmentation: OWL-ViT answers "where are the eggs," and SAM answers "what exactly are their pixel boundaries."

A lightweight **fallback mechanism** supplements primary OWL-ViT detections: when the primary detector's output is missing or low-confidence for an image, a fallback box-proposal step (`source: fallback-1` in `review_list.txt`) supplies a substitute box, and the image is automatically flagged for manual review so this substitution is never silent.

1.3 Dataset and Train/Eval Split

The dataset (Roboflow export, egg images across varied backgrounds — hands, trays, marble, cloth, keyboards, leaves, etc.) contains 110 total images after de-duplication of Roboflow's augmented copies.

To prevent data leakage (Roboflow's augmentation pipeline can produce multiple near-duplicate copies of the same original photograph), images were grouped by **original source photo** before splitting, and a 4:1 grouped split was applied:

- Total images: **110**
- Dev set: **88** images
- Eval set: **22** images

This exact split (recorded in `task1_outputs_final/split.csv`) is reused unchanged in Task 2, so the classical algorithm is developed and evaluated on the same images as the pretrained pipeline, making the Task 3 comparison fair.

1.4 Outputs

For each image, the pipeline produces:

- `masks/` — binary instance mask(s), one per detected egg
- `labels/` — class/instance label files
- `instances/` — per-instance crops/metadata
- `overlays/` — the original image with mask(s) drawn on top, for visual QA
- `diagnostics/` + `review_list.txt` — images flagged for manual review (fallback detections or manually-corrected boxes, `source: fallback-1` / `source: manual`)
- `results.csv` — per-image summary

1.5 Example Results

Figure 1 (clean result):

Figure 1: A clean, tight segmentation - OWL-ViT correctly localizes the egg held in a hand, and SAM produces an accurate mask over it.

Figure 2 (occlusion artifact):

Figure 2: The egg is partially occluded by a crossing plant stem. SAM's mask boundary is disrupted at the occlusion point, illustrating a minor limitation under occlusion.

Figure 3 (fallback detection, flagged):

Figure 3: Despite a visually clean result, this image's box came from the fallback detector rather than primary OWL-ViT output, and was automatically flagged in review_list.txt.

Figure 4 (unflagged failure):

Figure 4: A genuine failure case - the segmented region sits on a fold of tissue paper, not an egg. Unlike Figure 3, this image was NOT flagged for review, a silent failure.

1.6 Notes and Limitations

- As a zero-shot pipeline, quality depends on OWL-ViT's detection recall/precision for the word "egg." A fallback mechanism catches some low-confidence cases and flags them for review (Figure 3), but is not a complete safety net: Figure 4 shows a confidently-wrong mask that passed through unflagged, meaning the review process does not catch 100% of real failures. Any downstream use of Task 1 outputs as "ground truth" (as in Task 3) should account for this.
- Multi-instance images (multiple eggs in frame) are handled naturally since OWL-ViT can return multiple boxes per image, each independently segmented by SAM — this differs materially from Task 2's classical pipeline, which has no native multi-instance handling (discussed in Task 3).
- No console logs/timing/threshold values were retained from the original run beyond the split statistics above; this section is written from the saved outputs (split.csv, review_list.txt, overlays) rather than run logs.

Section 2: Task 2 - Classical Image-Processing Egg Segmentation

2.1 Objective

The goal of Task 2 was to build a **classical (non-deep-learning) image-processing algorithm** that segments eggs as closely as possible to the quality of Task 1's pretrained OWL-ViT + SAM pipeline, using only traditional computer vision techniques (thresholding, edge detection, color-space operations, morphological operations, etc.) — no neural networks, no learned weights.

To keep the comparison in Task 3 fair, Task 2 reuses **the exact same dev/eval split** established in Task 1 (`task1_outputs_final/split.csv`: 110 total images, dev=88, eval=22). All development and trial-and-error below was performed on the **dev set only**; the eval set was not used to tune any parameter, and the final locked-in algorithm was run once over the full 110-image dataset for reporting.

It was expected going in — and is confirmed by Task 3's quantitative analysis — that a classical pipeline would **not** match SAM's pixel-level accuracy. The goal was to get as close as a classical approach reasonably can, build genuine intuition about why it falls short, and document that gap rigorously. That gap is the subject of Task 3.

2.2 Experiment Log

Nine trials were run in sequence on the dev set, each building on diagnosis of the previous trial's failures. The full log — including abandoned approaches — is reproduced below.

Trial	Change	Why Tried	Dev-Set Result	Verdict
1	Global Otsu thresholding on grayscale image	Simplest possible baseline — test if a single global threshold separates egg from background	Worked only on a handful of high-contrast, plain-background images; failed when background brightness overlapped egg brightness	Discarded — too simplistic, no spatial reasoning
2	Otsu + morphological clean-up + "pick most egg-shaped blob" scorer	Add shape-based selection to reject non-egg-shaped blobs from noisy segmentation	Marginal improvement; threshold itself remained the bottleneck	Discarded — downstream filtering can't fix an unreliable upstream threshold
3	Canny edge detection to contour extraction to ellipse-fit scoring (RETR_EXTERNAL)	Eggs are elliptical; edge+ellipse fitting should be more lighting-robust than intensity thresholding	Found edges often, but shadows/texture created spurious contours; false "egg-shaped" picks on clutter	Discarded as-is, iterated further (3b-3e)

3b	Switched to RETR_CCOMP to also search holes/internal contours	Hypothesis: true egg boundary sometimes appears as an internal contour	Recovered a few missed cases, introduced new false positives from internal texture (cloth folds, marble veining)	Discarded — net neutral, instability increased
3c	Added relative-size filter on contours	Prune obviously wrong-scale false candidates from 3b	Removed some false positives, but filtered out legitimate eggs at unusual scales	Discarded — couldn't separate true-but-odd-scale from false-but-plausible-scale
3d	Added absolute HSV brightness filter	Exploit that eggs are often lighter than many backgrounds	Worked on tuning images only; broke on darker eggs / bright backgrounds (skin, pale tile, keyboard highlights)	Discarded — absolute thresholds don't generalize across lighting diversity
3e	Switched to relative brightness/uniformity filter	Make the brightness heuristic lighting-invariant	Better than 3d but still failed on ash/rock textures, hand creases, keyboard gaps, patterned cloth	Discarded — confirmed local shape+color+brightness heuristics cannot generalize across this dataset
4	Combined: accept if EITHER Otsu blob OR Canny/ellipse contour scored well	Hypothesis: the two methods fail on different images; combining covers more cases	Some images improved, but combining two brittle methods mostly combined their failure modes (more false positives)	Discarded
5	Lighter variant of Trial 4: tuned closing + combined scoring	Last attempt before abandoning the family — rule out that a lighter touch fixes brittleness	Continued failing on same categories as 3e; each fix for one image broke a previously-working one	**Entire contour/shape-scoring family abandoned.** Root cause: local heuristics are fundamentally brittle and cannot generalize across this dataset's background diversity
6	GrabCut seeded with plain centered rectangle + YCrCb skin-removal post-step	GrabCut models global fg/bg color distributions — fundamentally different from local shape heuristics	1/8 clean, 1/8 near-success; 2/8 partial failures from skin-removal over-firing on pale egg highlights; 3-4/8 severe background bleed; 1/8 total fg/bg inversion on patterned cloth	Kept concept; diagnosed two bugs: (A) skin mask too broad, catches pale egg highlights; (B) plain rectangle seed gives no guaranteed-fg anchor, allows inversion

7	Trimap seeding with hard GC_FGD center core + saturation-gated skin removal	Directly address both Trial 6 bugs	Net regression — 0/8 improved, several much worse; one hand image produced a totally unrelated diagonal wedge mask	Discarded — hard FGD core poisons the color model when background pixels fall inside the box (nearly always); largest-component pick can select an unrelated object if skin removal fragments the true foreground
8	Soft trimap (hard GC_BGD border only, no hard foreground) + one-time retry if fg >55% of frame + saturation-gated skin removal + pick component closest to image center (not largest)	Fix Trial 7's two bugs directly	Substantial improvement — most images now produce a plausible single egg-centered region; bleed reduced but not eliminated; residual thin "neck" connections remained	**Kept** — best result so far, neck-shaped bleed artifacts remained
9	Added morphological opening (kernel approx 2% of image diagonal) to sever thin necks, then re-apply centroid-pick and hole-filling	Opening removes thin connecting structures while preserving the wider main blob	Fixed several neck-connected cases cleanly; confirmed opening did NOT sever wide/stubby necks on marble, ash/rock, steel-pan backgrounds; widening kernel further risked eroding smaller eggs	**Locked in as final.** Accepted as a real technique ceiling: opening helps thin-neck bleed but not wide-neck bleed

Trial budget note: after Trial 7's regression, a hard budget of 2-3 further trials was agreed before locking in whatever variant performed best. Trial 9 was the final trial under that budget.

2.3 Final Algorithm (Locked In)

The final pipeline, implemented in `task2_segment_eggs.py`, is:

1. **Soft-trimap-seeded GrabCut** — only the border region outside a detection box is marked hard background (GC_BGD); everything inside is left as "probable" fg/bg for GrabCut to resolve iteratively. No hard foreground core is used.
2. **One-time adaptive retry** — if the resulting foreground covers more than 55% of the frame, the box is tightened and GrabCut is re-run once.
3. **Saturation-gated YCrCb skin removal** — skin-colored pixels are removed from the foreground only above a saturation threshold, protecting pale egg highlights.
4. **Morphological opening** (kernel approx 2% of image diagonal) to break thin background-bleed "necks."
5. **Centroid-based component selection** — the connected component whose centroid is closest to the image center is kept, not the largest by area.
6. **Hole-filling** to produce a clean, solid final mask.

Output structure mirrors Task 1 (`task2_outputs_final/masks/`, `labels/`, `instances/`, `overlays/`, `results.csv`, `review_list.txt`), with `results.csv` additionally logging `area`, `frame_frac`, `retried`, `flagged`, and `flag_reason` per image for transparency.

2.4 Full-Dataset Results

The final script was run once over the complete 110-image dataset (both dev and eval), with no crashes:

- **110/110 images processed successfully**
- **39/110 (35.5%) flagged for review**, predominantly due to suspected background bleed
- **2/110 complete failures** (area = 0): a three-eggs-on-marble scene, and a hand+egg scene on low-contrast tile floor
- An additional confirmed qualitative failure: a foil snack packet fused with the egg through a thick color-similarity "neck," surviving the morphological opening step due to its width

2.5 Known Limitations

- **No native multi-instance handling.** The pipeline selects exactly one connected component per image by design. Scenes with multiple eggs will have at most one correctly captured — a structural limitation, and the dominant root cause behind several of Task 3's failure categories (see Section 3.3.1).
- **Global color-model bleed.** GrabCut's foreground/background color model is global per image; when the egg's color is statistically similar to a large part of the background, the foreground region can bleed into or fuse with it. The morphological-opening fix only resolves thin-neck cases; wide necks are a confirmed, accepted failure mode.
- **No semantic understanding of "egg."** Unlike Task 1's OWL-ViT detector, the classical pipeline has no concept of what an egg looks like — it can confidently segment a non-egg object if that object's color statistics happen to separate well from its local surroundings.
- Parameter choices were tuned on the dev set by visual inspection, not formal grid search or cross-validation — consistent with the trial-and-error approach documented above.

Section 3: Task 3 - Quantitative and Qualitative Evaluation

3.1 Methodology

Task 2's classical segmentation outputs were evaluated against Task 1's SAM-based outputs, treated as ground truth. For each of the 110 images, the corresponding Task 1 mask and Task 2 mask were paired by filename stem and compared pixel-by-pixel using standard segmentation metrics: Intersection-over-Union (IoU), Dice coefficient, precision, and recall.

Mask pairing bug, found and fixed. The original evaluation script's image-lookup function assumed a flat folder structure, but the dataset's actual Roboflow "Folder" export nests images one level deeper. This caused the visualization step's image lookup to silently fail for every image, producing an empty worst_cases folder while the script still printed a success message. Critically, this bug was isolated to the visualization lookup only — the core metrics computation loaded masks directly by filename via dictionary lookups, reported zero missing files throughout, and was unaffected. The bug was fixed by switching to a recursive file search, and the resulting worst-case comparison images were manually inspected to confirm all reported failures were genuine.

3.1.1 Verifying failures were real, not a scripting artifact. Before accepting the quantitative results, a specific concern was investigated: several of the worst-scoring images showed precision = 0 and recall = 0 simultaneously, which could indicate either (a) a genuine wrong-region segmentation, or (b) a mask-pairing/indexing bug. After fixing the visualization bug and inspecting the actual image pairs, this was conclusively resolved as **(a): real failures.** In every zero-IoU case inspected, the Task 1 ground-truth mask was a clean, correctly-placed mask on the actual egg, while the Task 2 prediction was a confidently-segmented but completely unrelated region (a printed packaging graphic, a tray rail, a fabric seam, a floor shadow, a cable, a printed label). This confirmed genuine wrong-object capture, not a script bug, and directly informed the disjoint_wrong_region failure category defined below.

3.2 Quantitative Results

Full-dataset results (110 images, 0 missing/unmatched):

```
IoU       mean=0.3436  median=0.2685  min=0.0000  max=0.9943
Dice      mean=0.4422  median=0.4234
Precision mean=0.4312  median=0.2873
Recall    mean=0.7217  median=0.9003

IoU >= 0.75 (good):      19 / 110  (17.3%)
IoU 0.5-0.75 (ok):        7 / 110  ( 6.4%)
IoU < 0.5 (poor/failed): 84 / 110  (76.4%)
```

Figure 5 — IoU distribution across the dataset:

Figure 5: IoU distribution - heavily left-skewed, most images score well below 0.5 IoU with a smaller cluster above 0.75.

The distribution is heavily left-skewed: the majority of images score well below 0.5 IoU, with a smaller cluster of genuinely strong results above 0.75. There is no meaningful middle ground — the algorithm tends to either work well or fail badly.

Figure 6 — Precision vs. recall per image:

Figure 6: Precision vs. recall scatter - recall is systematically higher than precision, indicating over-segmentation as the dominant error mode.

The mean recall (0.72) is substantially higher than mean precision (0.43), and the median recall (0.90) is far higher than median precision (0.29). This asymmetry indicates the classical pipeline's dominant error mode is **over-segmentation** rather than under-segmentation, matching the GrabCut background-bleed behavior documented in Task 2.

3.3 Failure-Mode Taxonomy

Each image was classified into exactly one of seven categories using a fixed rule set applied in priority order (pass, total_miss, disjoint_wrong_region, mislocalized, bleed_overseg, partial_underseg, mixed_moderate):

```
pass                   26 ( 23.6%)  mean_iou=0.833
total_miss              2 (  1.8%)  mean_iou=0.000
disjoint_wrong_region  13 ( 11.8%)  mean_iou=0.000
mislocalized            5 (  4.5%)  mean_iou=0.119
bleed_overseg          56 ( 50.9%)  mean_iou=0.228
partial_underseg        7 (  6.4%)  mean_iou=0.339
mixed_moderate          1 (  0.9%)  mean_iou=0.415
```

Figure 7 — Category counts:

Figure 7: Count of images in each failure-mode category. bleed_overseg dominates at 50.9% of the dataset.

Figure 8 — Predicted/GT area ratio vs. IoU:

Figure 8: Predicted/ground-truth area ratio vs. IoU. bleed_overseg cases cluster at high area ratios, confirming over-segmentation.

`bleed_overseg` is the single dominant failure mode at 50.9% of the dataset, consistent with the precision/recall asymmetry in Section 3.2. Its median predicted-area/ground-truth-area ratio of 4.21 confirms these predictions are, on average, over four times larger than the true egg region — direct

numerical evidence of background bleed.

3.3.1 Root-cause separation via multi-instance analysis

The seven raw categories above describe symptoms, not root causes. Each image's Task 1 ground-truth mask was checked for multi-instance content (2+ connected components, each 3%+ of mask area), since Task 2's single-component-picking design is known to be structurally blind to multi-egg scenes.

Category	N	Multi-instance	Multi-instance rate
pass	26	3	11.5%
total_miss	2	1	50.0%
disjoint_wrong_region	13	1	7.7%
mislocalized	5	3	60.0%
bleed_overseg	56	3	5.4%
partial_underseg	7	7	100.0%
mixed_moderate	1	0	0.0%

Dataset baseline multi-instance rate: 16.4% (18/110)

This resolves the seven categories into **two independent root causes**:

- **Root Cause A — single-object color-model failure** (bleed_overseg + disjoint_wrong_region = 69 images, 62.7% of the dataset): GrabCut's global per-image color model either over-grows the foreground onto similarly-colored background (bleed) or latches onto an entirely different salient region (wrong-region capture). Both categories occur at or below the dataset's baseline multi-instance rate (5.4% and 7.7% vs. 16.4% baseline), confirming they are **not** explained by multi-egg scenes.
- **Root Cause B — no multi-instance handling** (partial_underseg fully, plus the majority of mislocalized/total_miss, approx 14 images, 12.7% of the dataset): partial_underseg is **100%** explained by multi-instance scenes — every image in this category contains multiple eggs, of which the algorithm can only capture one. mislocalized (60%) and total_miss (50%) are majority multi-instance-driven as well, with a minority of genuine single-egg exceptions.

3.4 Qualitative Analysis

Nine representative examples were hand-selected across categories for close visual inspection, in addition to the 10 zero-IoU worst-case images already reviewed in Section 3.1.1.

Root Cause A examples (bleed / wrong-region)

Figure 9: not_damaged_43 (IoU=0.269, P=0.291, R=0.782) - the egg is captured cleanly (yellow overlap), but a large red false-positive region extends across the laptop body via a shared dark-color neck, illustrating the wide-neck bleed ceiling from Trial 9.

Figure 10: not_damaged_135 (IoU=0.057, P=0.057, R=1.000) - recall of 1.0 confirms the true egg region is fully contained within the prediction, but precision of 0.057 shows it is swamped by an enormous false-positive region (a bead-filled bucket).

Figure 11: not_damaged_123 (IoU=0.000) - the prediction (red) sits entirely on an egg tray's plastic rail, while the true egg (green) is untouched - a complete wrong-object capture.

Root Cause B examples (multi-instance)

Figure 12: not_damaged_114 (IoU=0.170, P=0.228, R=0.400) - a multi-egg marble scene. One egg is partially captured (yellow), while the remaining eggs are entirely untouched (pure green).

Figure 13: not_damaged_133 (IoU=0.322, P=0.995, R=0.322) - precision near-1.0 shows the captured region is a clean, accurate segmentation of the one egg it selected; low recall is entirely due to other eggs being missed.

Passing-case contrast examples

Figure 14: not_damaged_34 (IoU=0.994) - near-pixel-perfect agreement with Task 1's SAM mask, confirming the classical pipeline can produce excellent results under favorable conditions.

Figure 15: not_damaged_161 (IoU=0.515) - a boundary case where the overall region is correct, but the bottom rim of the egg is slightly missed, likely due to a shadow being excluded by GrabCut's color model.

3.5 Discussion and Suggested Improvements

The quantitative and qualitative analysis converges on two independent, well-evidenced root causes behind Task 2's gap relative to Task 1:

For Root Cause A (single-object color-model failure, 62.7% of the dataset): GrabCut's global color-distribution model has no notion of object boundaries beyond color/contrast statistics. A practical improvement would be an **edge-aware refinement pass** (using detected object boundaries or a learned saliency/objectness prior to constrain the foreground region, rather than relying purely on global color separation). This would target both bleed and wrong-region capture, without requiring full deep-learning detection.

For Root Cause B (no multi-instance handling, 12.7% of the dataset): The current design deliberately picks exactly one connected component per image. A straightforward fix — **retaining all sufficiently large connected components instead of a single centroid-pick** — would allow the algorithm to output multiple egg instances per image where present, directly eliminating the partial_underseg category and likely improving a majority of mislocalized and total_miss cases as

well.

Both improvements are classical (non-deep-learning) in nature and were considered out of scope for the trial budget agreed during Task 2's development, but are flagged here as the clear next steps if further iteration were pursued.